

INSTITUTE OF ENGINEERING & MANAGEMENT, KOLKATA

Department of Computer Science & Engineering

CAPSTONE PROJECT

SELENIUM PYTHON AUTOMATION FRAMEWORK

A Scalable Web UI Testing Framework for E-Commerce Applications

PROJECT FOCUS

Automated Login & Product Search Testing

Selenium WebDriver • Python • PyTest • Unittest • POM • CSV Data-Driven Testing

PROJECT REPORT

Prepared by

RAMALA SAMANTA

B.Tech — Computer Science & Engineering

7th Semester | Academic Session 2026–2027

University of Engineering and Management, Kolkata

September 2026

Table of Contents

- 1. Abstract**
- 2. Introduction**
- 3. Scope of the Project**
- 4. Tools and Technologies Used**
- 5. Framework Architecture**
- 6. Test Design and Methodology**
- 7. Implementation Details**
- 8. Test Execution and Results**
- 9. Challenges Faced and Solutions**
- 10. Conclusion**
- 11. Future Enhancements**
- 12. References / Bibliography**

Abstract

This project focuses on the design and development of a scalable web automation testing framework using **Python and Selenium WebDriver**. The framework is designed to automate critical user journeys of a live e-commerce application, with a primary focus on **user login and product search** functionalities. The project demonstrates how modern test automation practices can be combined to create a structured, reusable, and maintainable testing solution.

The framework follows the **Page Object Model (POM)** design pattern to separate web page interaction logic from test execution logic, improving code organization and reducing duplication. Both **Unittest and PyTest** frameworks are implemented to demonstrate different approaches to automated test execution. The framework also incorporates **data-driven testing** using CSV files, enabling multiple sets of test data to be executed without modifying the core test scripts. Configuration details are maintained externally to support portability and execution across different environments.

To improve test analysis and debugging, the framework generates **HTML test reports** and captures screenshots whenever a test case fails. These features provide clear visibility into test execution results and help identify the cause of failures. Overall, the project demonstrates the practical application of Selenium WebDriver and software testing principles in building a professional-style automation framework that can be extended to support additional test scenarios and application modules.

1. Introduction

1.1 Background

Manual testing of web applications can be time-consuming, repetitive, and susceptible to human error, particularly for regression-intensive workflows such as user login and product search that need to be verified after every software release. Test automation addresses these challenges by enabling the same test steps to be executed repeatedly, consistently, and efficiently.

Selenium WebDriver is a widely used open-source framework for browser automation. When combined with a well-designed automation architecture, Selenium can provide a reusable and maintainable testing solution rather than a collection of independent test scripts. A structured automation framework also improves test execution, reporting, debugging, and future scalability.

1.2 Objective

- The primary objective of this capstone project was to design and develop a robust **Selenium Python automation framework** incorporating industry-relevant automation and testing practices. The framework includes the following components:
- **Unittest** – Python's built-in testing framework for organizing and executing test cases.
- **PyTest** – A flexible and feature-rich testing framework widely used for Python-based automation.
- **Page Object Model (POM)** – A design pattern that separates web-page locators and interaction methods from test logic.
- **Utility Classes** – Reusable helper modules for common operations and reducing code duplication.
- **Configuration Management** – Externalized configuration settings that make the framework easier to maintain and execute across different environments.
- **CSV-Based Test Data Handling** – Supports data-driven testing by allowing multiple test inputs to be maintained separately from the test scripts.
- **Screenshot Capture on Failure** – Automatically captures screenshots when a test fails to support debugging and failure investigation.
- **HTML Reporting** – Generates readable and shareable reports containing test execution results.

1.3 Project Scenario

The framework automates the Login and Product Search functionality of `automationexercise.com`, a publicly available e-commerce practice website. These two features were chosen because they represent the most common entry points into any e-commerce application — authenticating a user and helping them find products — and each has a rich set of both positive and negative test scenarios worth automating.

2. Scope of the Project

The scope of this project focuses on developing and implementing a structured Selenium WebDriver automation framework for functional testing of an e-commerce web application. The major areas covered by the project are as follows:

- **Login Functionality:** Automated validation of the Login page using six test scenarios, including valid credentials, incorrect password, unregistered user, and three different empty-field combinations.
- **Product Search Functionality:** Automated testing of the Product Search feature using both valid keywords that return matching products and keywords that return no search results.
- **Multiple Test Runners:** Support for both **Unittest** and **PyTest**, allowing the same underlying automation framework to be executed using different testing frameworks.
- **Multi-Browser Support:** The framework supports **Google Chrome, Mozilla Firefox, and Microsoft Edge**, with browser selection controlled through external configuration.
- **Headless Execution:** The framework supports headless browser execution, allowing automated tests to run without opening the browser's graphical interface.
- **Configuration Management:** Browser selection and other execution-related settings are maintained externally, making the framework easier to configure and portable across different environments.

The project is limited to **UI-level functional automation testing**. It does not cover performance testing, API testing, security testing, or database validation. The primary focus is on creating a **reusable, maintainable, and scalable Selenium automation framework** capable of supporting reliable functional and regression testing.

3. Tools and Technologies Used

Category	Tool / Technology
Programming Language	Python 3.14
Browser Automation	Selenium WebDriver 4
Test Frameworks	unittest (built-in), PyTest 9.x
Reporting	pytest-html
Test Data Format	CSV
Configuration	configparser (config.ini) + environment variables
Version Control	Git and GitHub
IDE	Visual Studio Code
Target Application	automationexercise.com
Operating System (dev)	macOS

4. Framework Architecture

4.1 Design Approach

The framework follows a layered architecture in which each layer has a single, well-defined responsibility. This separation means a change to the application's UI only requires updating the corresponding page object, not every test that touches that page.

Layer	Responsibility
Tests (tests/)	Business-level assertions; what should happen, not how to click it
Page Objects (pages/)	Locators and page-specific actions (login(), search())
Utilities (utilities/)	Cross-cutting helpers: config, driver setup, CSV reading, screenshots, logging
Test Data (testdata/)	CSV files holding scenario inputs and expected outcomes
Configuration (config/)	Environment-specific settings, kept out of test code

4.2 Directory Structure

```
selenium_framework/
├── config/config.ini           # URL, browser, headless, waits, credentials
├── pages/                     # Page Object Model
│   ├── base_page.py          # shared Selenium wrappers
│   ├── home_page.py
│   ├── login_page.py
│   └── products_page.py
├── utilities/
│   ├── config_reader.py       # config.ini + env-var overrides
│   ├── driver_factory.py      # Chrome / Firefox / Edge, headless
│   ├── csv_reader.py          # data-driven test data
│   ├── screenshot.py          # screenshots on failure
│   └── logger.py              # console + file logging
├── testdata/                  # login_data.csv, search_data.csv
├── tests/
│   ├── base_test.py           # unittest base (setUp/tearDown)
│   ├── test_login_unittest.py
│   ├── test_search_unittest.py
│   ├── test_login_pytest.py
│   └── test_search_pytest.py
├── conftest.py                # pytest fixtures + failure-screenshot hook
├── pytest.ini                 # markers + HTML report config
├── run_unittest.py            # standalone unittest runner
├── reports/
├── screenshots/
└── logs/
```

4.3 Page Object Model (POM)

Every page of the application under test is represented by a Python class inheriting from a shared `BasePage` class. `BasePage` wraps common Selenium operations — click, type, wait for visibility, wait for title — with explicit waits (`WebDriverWait`) rather than fixed sleeps, and includes a JavaScript-click fallback for elements intercepted by overlays or ads. Concrete page classes such as `LoginPage` and `ProductsPage` define only their own locators and page-specific actions (`login()`, `search()`, `is_logged_in()`). Tests interact exclusively through these methods and never reference a locator directly, which keeps the tests readable and isolates UI changes to a single file.

5. Test Design and Methodology

5.1 Data-Driven Testing

Rather than hardcoding input values inside test methods, all scenario data is stored in CSV files under `testdata/`. Each test iterates over the rows of the relevant CSV — through PyTest's `@parametrize` decorator, or through unittest's `subTest()` context manager — so adding a new scenario is a matter of adding a row, not writing new code.

5.2 Login Test Scenarios

Test Case	Input	Expected Result
valid_login	Registered email + correct password	Login succeeds
wrong_password	Registered email + incorrect password	Login fails, error message shown
unregistered_user	Unregistered email + any password	Login fails, error message shown
empty_email	Blank email + password	Blocked by field validation
empty_password	Email + blank password	Blocked by field validation
empty_both	Both fields blank	Blocked by field validation

5.3 Product Search Scenarios

Test Case	Keyword	Expected Result
search_tshirt	tshirt	Results returned
search_dress	dress	Results returned
search_jeans	jeans	Results returned
search_top	top	Results returned
search_invalid	xyzabc12345	Zero results, no error

6. Implementation Details

6.1 Configuration Management

All environment-specific values — base URL, browser choice, headless flag, explicit wait duration, and credentials — are stored in `config/config.ini` and read through a single `ConfigReader` utility class. The reader checks environment variables first (e.g. `BROWSER`, `HEADLESS`, `VALID_EMAIL`, `VALID_PASSWORD`) before falling back to the file, allowing the same codebase to run locally and in a CI pipeline without modification, and keeping real credentials out of version control.

6.2 Driver Management

A `driver_factory` module centralizes `WebDriver` creation. Based on config, it launches Chrome, Firefox, or Edge, with or without headless mode, using Selenium 4's built-in driver management so no manual driver binaries need to be downloaded or maintained.

6.3 Screenshot on Failure

Failure diagnostics are implemented at two levels. Under PyTest, a `pytest_runtest_makereport` hook in `conftest.py` inspects every test outcome; on failure, it retrieves the active `WebDriver` instance, saves a timestamped PNG to `screenshots/`, and embeds the image directly into the `pytest-html` report. Under plain unittest, the same capture logic runs from `BaseTest.tearDown()`, which inspects the test's outcome and triggers a screenshot independently of PyTest's hook. This dual implementation ensures failure evidence is captured no matter which runner executed the test.

6.4 Logging

A shared logger utility writes timestamped, leveled log messages to both the console and a per-day log file under `logs/`. Every page action (opening a URL, typing into a field, clicking a button) is logged, which turns a failing test from a bare stack trace into a readable sequence of what the framework actually did before it failed.

6.5 HTML Reporting

Test execution is reported using the `pytest-html` plugin, configured through `pytest.ini`. Each report includes a pass/fail summary, per-test duration, environment metadata (Python version, platform, installed plugin versions), and — for any failure — the embedded screenshot and captured log output, all in a single self-contained HTML file.

7. Test Execution and Results

7.1 Execution Summary

The complete suite was executed against the live application using both PyTest and unittest entry points. A representative run produced the following summary:

Metric	Result
Total tests collected	18
Passed	18
Failed	0
Subtests passed (unittest)	11
Total execution time	≈ 2 minutes 6 seconds
Browser	Chrome (non-headless)

7.2 Sample Test Run Output

```
tests/test_login_pytest.py::test_login_page_title PASSED
tests/test_login_pytest.py::test_login_data_driven[valid_login] PASSED
tests/test_login_pytest.py::test_login_data_driven[wrong_password] PASSED
tests/test_search_pytest.py::test_product_search[search_dress] PASSED
tests/test_search_pytest.py::test_product_search[search_jeans] PASSED
...
===== 18 passed, 11 subtests passed in 125.96s (0:02:05) =====
```

7.3 Report Artifacts Generated

- reports/report.html — full HTML test report with pass/fail breakdown
- screenshots/ — PNG captured automatically for any failed test
- logs/automation_<date>.log — full step-by-step execution log

8. Challenges Faced and Solutions

Challenge	Solution
Title assertion occasionally failed because the page title had not yet updated when checked immediately after navigation.	Added an explicit <code>wait_for_title_contains()</code> helper using <code>WebDriverWait</code> with <code>expected_conditions.title_contains()</code> , instead of reading the title immediately.
One search test intermittently timed out waiting for the results heading, due to slower response times from the live site under load.	Increased the configurable <code>explicit_wait</code> value in <code>config.ini</code> , giving all waits more headroom without hardcoding sleeps.
An assertion that every search result's name contained the search keyword failed, because the site's search also matches category and description text.	Relaxed the assertion to check that at least one result name contains the keyword, matching the site's actual search behavior.
Keeping real login credentials out of the public GitHub repository.	Externalized credentials to <code>config.ini</code> with placeholder defaults, and supported <code>VALID_EMAIL</code> / <code>VALID_PASSWORD</code> environment variable overrides for real values.

9. Conclusion

This project successfully delivers a Selenium Python automation framework that meets every objective set out in the assignment brief: dual support for Unittest and PyTest, a clean Page Object Model separating locators from test logic, reusable utility classes, externalized configuration, CSV-driven test data, automatic screenshots on failure, and HTML reporting. Beyond satisfying the checklist, the project also surfaced and resolved real-world automation challenges — timing flakiness, live-site response variability, and credential security — that mirror the kind of problems encountered in professional QA automation work. The resulting framework is not a fixed set of scripts but a reusable foundation: new test scenarios can be added purely as CSV rows, new pages can be added by extending BasePage, and the framework can be pointed at a different environment by editing a single configuration file.

10. Future Enhancements

- Continuous Integration pipeline (GitHub Actions or Jenkins) to run the suite automatically on every push.
- Parallel test execution using pytest-xdist to reduce total run time.
- Selenium Grid or remote WebDriver support for cross-machine and cross-browser execution.
- Retry logic for assertions affected by live-site network variability.
- Extending coverage to additional flows such as cart, checkout, and user registration.

11. Bibliography

- Selenium WebDriver Documentation — <https://www.selenium.dev/documentation/>
- PyTest Documentation — <https://docs.pytest.org/>
- Python unittest Documentation — <https://docs.python.org/3/library/unittest.html>
- pytest-html Plugin — <https://pytest-html.readthedocs.io/>
- Target Application Under Test — <https://automationexercise.com>
- Project Source Code (GitHub) — <https://github.com/elysian-07/selenium-python-framework->